Code Redstart: Introduction to Computer Programming

Copyright: Andrew Vella, 2026. MIT License.

https://www.coderedstart.com

The paramount goal of this crash course is to demonstrate that the computer is a machine you can use, and own. Modern computers are designed to make us forget the machine is a machine, and not a magical device, like a palantir from Lord of the Rings. Computers have a history, and a logic, that you can understand. Why should you learn programming? So you can use your computer without all of those noisy apps. Know the machine, own the machine, save money on apps and subscriptions for the machine.

This course features 30 assignments designed to practice the fundamentals of programming, from your first "Hello World!", through loops, functions, and structures, up to animating your own graphics. Here are a few good principles to keep in mind while programming:

- Keep it simple
    - Do not write code for vague issues. Write code for the task at hand. Programming is the last thing you do.
- Keep it clean
    - Errors happen, and you can handle them. Do so, one at a time.
- Save your work
    - You will make a typo that breaks everything. That is ok, if you have a copy of the work before that typo.
- Separate Tasks: Divide and Conquer
    - Do not write one big program for one big task. Write small programs for small tasks, and then use them to do the big task.
- Read the Docs
    - Someone has encountered your problem before. Learn how they solved it, and apply what you learned to your own work.
- Comment the Whys
    - Do not describe what your code does. Describe why you wrote it. Delete code that does not pass the "why" test.
- Stuck? Take a break
    - Sometimes the problem is easier to solve after you get up, have a snack, drink water, and come back. You do not have to act like a computer.

The answer key is written largely in python, with a bit of html. I did my best to implement everything myself, out of fairness to you. That said, there are plenty of websites with open source code to read, like GeeksforGeeks and W3. There are also plenty of good videos out there. This did not all come right off the dome!

Table of Contents

- Getting Started with Python
  - Installation
  - Running Code
- Sources and Resources
  - Online Documentation
  - Books
  - Integrated Development Environment
  - Text Editors
  - Video Creators
- A - Hello Worlds
  - Hello World
  - About Me
  - Funny Fill-in
  - Turtle
- B - Conditionals, Loops, and Variables
  - Babbling Baby
  - Fizz Buzz
  - Guess the Number
  - Rock Paper Scissors
- C - Arrays and Functions
  - Array Generator
  - Linear Search
  - Prime Number Generator
  - Pascal's Triangle
- D - Recursion
  - Factorial
  - Binary Search
  - Number Namer
  - Sirpinski's Triangle
  - Dragon Curve
- E - Sorting Algorithms
  - Bubble Sort
  - Merge Sort
  - Quick Sort
  - Bogo Sort
- F - Files and Error Handling
  - Calculator
  - Name Shuffler
  - Name Sorter
- G - Data Structures
  - Game of Telephone
  - Game of War
  - BInary Tree Traversal
  - Heap Sort
- H - Comparing Algorithms
  - Search Compare
  - Sort Compare
- I - Animation and Automata
  - Langton's Ant
  - Sort Visualizer
  - Game of Life
- Acknowledgements

Getting Started with Python

So the great thing about python is that the installer is nice to us.

Click this link and then click the big yellow download button.
https://www.python.org/downloads/

Launch the installer. Done.

Running Code

You will get an app called IDLE, which is an integrated development environment for python. You will also get a python shell, which is a window that can run lines of python code. IDLE is good for dealing with tab errors, also called indentation errors. (Sometimes tabs get mixed up with spaces and confuse the compiler.) To resolve these errors, you can "untabify" your code in IDLE.

To start, create a folder for your python scripts, and then create a new document in that folder. You can use IDLE, or any text editor to create the doc, just be sure it ends with ".py" for python.

You can open any .py file in IDLE and run it. If you prefer not to use IDLE you can choose another IDE like Pycharm, or you can run python from the command line pretty easily. Once you have opened the command line in your folder for python files (usually you would do this with a right click) you can type the following command:

```
python ./your_file.py
```

Sources and Resources

Online Documentation

- Docs https://docs.python.org/3/
- GFG - https://www.geeksforgeeks.org/python/python-programming-language-tutorial/
- W3 - https://www.w3schools.com/python/default.asp
- Python Online - https://www.online-python.com/
- Turtle- https://docs.python.org/3/library/turtle.html
- Colors - https://cs111.wellesley.edu/reference/colors

Books

- Think Python: How to Think Like a Computer Scientist by Allen B. Downey
- Python Cookbook: Recipes for Mastering Python 3 (3rd Edition) by David Beazley and Brian K. Jones
- Python Programming: An Introduction to Computer Science (4th Edition) by John Zelle

Integrated Development Environments

- IDLE - https://docs.python.org/3/library/idle.html
- Pycharm - https://www.jetbrains.com/pycharm/
- Spyder - https://www.spyder-ide.org/

Text Editors

- Atom - https://atom-editor.cc/
- Emacs - https://www.gnu.org/software/emacs/
- NeoVim - https://neovim.io/
- Nano - https://www.nano-editor.org
- Sublime Text - https://www.sublimetext.com/3

Video Creators:

- Computerphile - https://www.youtube.com/@Computerphile
- CC CS - https://www.youtube.com/playlist?list=PL8dPuuaLjXtNlUrzyH5r6jN9ulIgZBpdo
- Kuvina - https://www.youtube.com/@Kuvina
- Tom7 - https://www.youtube.com/@tom7
- Timo Bingmann - https://www.youtube.com/@TimoBingmann
- Uniprod - https://www.udiprod.com/

SECTION A - Hello Worlds

This section is all about putting things on the screen. At this stage, the goal is just to get the computer to show you something you made.

Hello World

- A standard introduction to any programming language, simply print "Hello World" to the screen.

About Me Page

- Make your own webpage in html, with a bio, image, and link.

Funny Fill-in

- Write your own funny story with blanks to fill in. Use `print()` and `input()`.

Turtle

- Use Python's turtle module to draw a star, smiley, or heart

SECTION B - Conditionals, Loops, and Variables

Now that you have learned output. It is time to learn to process input. These programs demonstrate important principles, like storing input with variables and then applying logic to produce an appropriate output. These conditional behaviors are how a computer actually computes. Loops will also be covered in this section. Loops let the computer repeat actions, like taking a prompt from the user. Conditionals are used to open and close loops. This section includes a few games to play, and an old mainstay called "Fizz Buzz".

Babbling Baby

- The babbling baby takes a question (input) and randomly answers with "yes", "no", or "i don't know" (or whatever else you like)

Fizz Buzz

- The Fizz Buzz program prints numbers with some substitutes. If a number is divisible by 3, print "Fizz". If a number is divisible by 5, print "Buzz". If a number is divisible by 3 and by 5, print "FizzBuzz"

Guess the Number

- The computer randomly picks a number between 1 and 10. You can guess until you guess correctly. If you guess low, the computer will print "Higher!". If you guess high, the computer will print "Lower!". The number of guesses you made will be displayed when you guess correctly

Rock Paper Scissors

- Implement the game "Rock Paper Scissors" and have the computer make a random selection. Then have the player pick an option. Use conditionals to determine wins, loses, and draws

SECTION C - Arrays and Functions

Arrays are groups of variables that can be used to organize data. Variables in an array are indexed. The first index in an array is usually called index 0. The number of indices an array has is called its length. An array of six variables has a length of five.

A function is a sub-program that can be run by a larger program. Functions are great because they allow you to describe your program in terms of smaller programs. These smaller programs can accept input, called arguments or parameters. Functions can also output, also called returning a value. You can run them from anywhere in your program, by making a function call. This saves a whole lot of debugging, and copy-paste. Functions can be tested independently from the rest of your program, so you can actually organize, test, and maintain code with these things.

Array Generator

- Write functions to generate an array of ordered numbers, one of shuffled numbers, and one of reversed order. IE: [1, 2, 3], [2, 1, 3], [3, 2, 1]. Have each function take an argument n, that specifies array length

Linear Search

- Write a search function that locates a target number in an array of numbers. Have it check each index in order until locating the target or completing a check on every index. If the search does not find anything, indicate this. Note: the index of an array refers to the position of an item in an array. The leftmost item is usually at index zero. This search should return (output) an index, or -1 if the target is not found.

Prime Number Generator

- Write a program that generates prime numbers out to a maximum value (n). A prime number is only evenly divisible by itself and one. (IE: 2, 3, 5, 7). Hint: Construct an array of primes using a loop and the modulus operator.

Pascal's triangle

- Write a function to generate lines of Pascal's triangle out to a specific number of iterations (an argument, n) After line two, each value on the current line is the sum of two values on the previous line. IE:

```
1
1  1
1  2  1
1  3  3  1
1  4  6  4  1
```

SECTION D - Recursion

Recursion is what happens when a function calls itself. It is great for doing stuff that can technically be done with loops, but is annoying to do with loops. Just be aware of the infinite recursion. Your program can only store so much on the call stack. Recursive functions often need a base case, to tell the function that it is time to stop calling itself. A recursive function can also have a loop in it, and often does.

Factorial

- Write a function that returns the factorial of a (natural) number. n factorial is equal to n * (n - 1) * (n - 2) * (n - 3)... to 1. Five factorial (5!) is 5 * 4 * 3 * 2 * 1. Use a recursive function, such that factorial(n) returns n * factorial(n - 1). The factorial of 1 is also 1. So factorial(1) can return 1 (called a base case).

Binary Search

- This is a search function that searches on an array sorted from least to greatest. You use this search when you are looking for a specific page in a book. You start in the middle, about halfway into the book. If your page is lower, you go to the first quarter of the book. If your page is higher, you go to the third quarter of the book. Then you check eighths of the book, closing in on your target page. Implement this search recursively. Return -1 if your target is not found, such as targeting page 300 in a 250 page book.

Number Namer

- This program takes a natural number, and outputs the english name of that number. 1 returns "one". 23 returns "twenty three". 154 returns "one hundred fifty four". Numbers beyond 9999 should return "ten thousand or greater" (or "one myriad or greater").

Sirpinski's Triangle

- Use turtle graphics to produce generations of Sirpinski's triangle. This is a self-similar fractal that looks a bit like the "Triforce" from "The Legend of Zelda", if each piece was made of smaller copies. Take an (upright) triangle, put an inverted triangle in the center. The inverted triangle is now surrounded by three smaller upright triangles. Put inverted triangles in the centers of these, and continue to recur to a chosen max; like n = 6.

Dragon Curve

- Use recursion and turtle graphics to produce generations of the Dragon Curve. This is a self-similar fractal composed of ninety degree turns. The first generation is a right turn. The next is a right, then a right, then a then a left. To get each new generation, start with two copies of the turn sequence from the previous generation. Take the first copy and append a right turn to the end of it. Take the second copy and reverse it. Then invert it such that rights become lefts and lefts become rights. Append this new sequence to the right turn that was appended to the first copy. Recur to a chosen max; like n = 12.

SECTION E - Sorting Algorithms

Okay, so a computer scientist walks into a library and finds a whole shelf has been knocked down onto the floor. Good Samaritans get the shelf back up and steady, but now a whole bunch of books need to be organized, without using anymore shelves. The computer scientist says "Ok, I think this book should be the pivot, or maybe we can put 'AAAAAA! The Book of Screams' on the shelf first." Sorting algorithms sort stuff, usually numbers, but they can sort words and other things too. They tend to perform quite well on stuff that has already been sorted, and still do good on stuff that is not sorted, but if you want to give them a real challenge, take a sorted list, and put it in reverse. (Sorted order generally means sorted from least to greatest. Greatest to least would be considered reverse order.)

Bubble Sort, Selection Sort, or Insertion Sort

- Write a function to sort a list of numbers. Do not concern efficiency yet. Choose between: Bubble Sort, which swaps pairs of adjacent numbers until sorted; Selection Sort, which selects the largest unsorted value, and puts in in sorted order; or Insertion Sort, which compares values to a key value, and then inserts the key into its sorted position. Your algorithm will almost certainly use a loop, nested inside a loop. Test your algorithm on a sorted array, a shuffled array, and a reversed array.

Merge Sort

- This is a sorting algorithm that uses a "divide and conquer" strategy. An array of numbers is split in half, then the halves are split in half, and so on until each part contains only one value. (The array is divided into many arrays with a single value each.) These arrays are then merged back together using `merge()` The original array is ultimately reconstructed in sorted order. Test your algorithm on a sorted array, a shuffled array, and a reversed array.

Quick Sort

- Quick sort uses a partition. A value on the array is chosen to be the pivot. This partitions the array into two partitions, called high and low. Everything higher than the pivot is sorted to the right, into the high partition, and everything less than the pivot is sorted to the left. Then another pivot is selected. Then each partition is also partitioned recursively, by halves, until partitions of only one value are produced. Test your algorithm on a sorted array, a shuffled array, and a reversed array.

Bogo Sort

- This is an impractical sorting algorithm. Fortunately, it does provide a lesson in verification functions (or verifiers). This algorithm checks if an array is sorted, and if the array is not sorted, the array gets shuffled until it is sorted. Test your algorithm on a sorted array, a shuffled array, and a reversed array. Keep your test arrays at a length greater than two, but less than five.

SECTION F - Files and Error Handling

So this section is all about managing program resources. If your program needs information from a file, how would you process that information? There are lots of ways to manipulate files (even dangerously). Instead of doing any damage, you are going to read to files, write to them, and deal with missing files. Further, you are going to write programs that are robust enough to handle bad input from users who write nonsense into the prompt.

Calculator

- Write a four function calculator that can do addition, subtraction, multiplication, and division. Ask the user for a number, operation, and number, then do the calculation and print it. Handle division by zero. Handle bad inputs, like "This is not a number", "Knight to e4", "3.213,234", or "@*&%". Also, round off insignificant figures.

Name Shuffler

- Read a list of names from a file "names.txt", shuffle the names and overwrite the file with the shuffled names. Handle a file that does not exist, like "thisisnotafile.txt". Observe your "names.txt" file before and after running this program. There is a "names.txt" file in the source code directory.

Name Sorter

- Read a list of names from a file "names.txt", sort the names and overwrite the file with the sorted names.Handle a file that does not exist, like "thisisnotafile.txt". Observe your "names.txt" file before and after running this program. There is a "names.txt" file in the source code directory.

SECTION G - Data Structures

A data structure is like a physical structure, except it is made of data instead. These are the puddings of computing, and a general term for "anything that is useful for organizing stuff in some way", even just throwing it all in a heap. So there are lots of data structures, like queues and stacks and trees and heaps. You can use object oriented programming to implement these (but Python is not great for that). Most of the time you can use arrays to implement these, if you prefer.

Game of Telephone

- A queue is a data structure with a simple rule, "first in, first out" or "first come, first serve" You have been in one of these every time you stand in line. Standing in line is boring, so this assignment simulates a game of Telephone instead. This is a game you may have played as a small child. Kids sit in a row and the first one gets to pick something to whisper in their friend's ear. Then the friend whispers something to the next kid. So it goes down the chain of kids until the last one proclaims something. In this simulation, the first kid gets a word, they run a mutating function on it, and then they pass it on to the next one, who mutates the word again. Each kid comes with some random parameters that determine what they will do. A kid may ignore the word they were given and randomly say something else. They may switch letters around, or swap them out for random letters. The word will be printed each time this happens. You may also give each kid a random name that prints alongside their mutation, and call them silly if they ignore the word they were given. You can use text files, or dictionaries, to store silly words and names.

Game of War

- A stack is a data structure with a simple rule "last in, first out". These work exactly like they do on your plate of pancakes. The pancake at the top gets plated last and eaten first. Stacks also describe decks of cards, (and lots of concepts in computer programming: function calls, memory, scopes...). This assignment implements a card game using stacks of cards, called War. This assignment pares the game down to a minimal rule set:
    - The deck is shuffled and dealt to two players, so each gets half the deck.
    - On each turn each player plays one card from the deck, and the two cards are compared to determine who gets a point.
    - Points are awarded according to the cards played in each turn. Cards have one of four suits: Clubs, Diamonds, Hearts, and Spades, in American Bridge Suit Order; and one of thirteen ranks: 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King, Ace. Aces beat Kings, Twos beat Aces, Threes beat Twos; no Jokers.
    - The higher ranking card wins the point for that player. If cards are of the same rank, the players go to war. When a war happens, the player with the higher suit wins. Spades beat Hearts, which beat Diamonds, which beat Clubs.

Binary Tree Traversal

- A Binary Tree is a data structure where nodes are arranged into parents and children. A node usually stores a number, but it can also store other things. A parent can have no children, one child, or two children, but no more. This rule set forms a tree-like structure, with a root that has no parents, and leaves that have no children. (The "root" is analogous to the trunk of an actual tree. It branches off into branches, which branch off into leaves) This structure is ideal for short-term storage, as every child can be accessed from the root of the tree, using a traversal. A traversal visits every node in the tree, from root to leaves, or leaves to root, left child to parent to right child. Any traversal is acceptable as long as every node in the tree is visited before the traversal completes. You can use an array to store your tree, recursion to traverse your tree, and a "visited" array to keep track of visited nodes during the traversal. (You might be thinking "Just use a table, so there is no traversing!" Although a binary tree needs to be traversed, this structure can store and free data easily, because there is always a leaf, ready to adopt a child, somewhere on the tree. A table may have faster look up times, but data cannot share a cell. Cells need to be specifically managed for storage.) Note: for a tree implemented as an array, the root is at index 0. To get the children of the node at index i, take index 2i + 1 for the left child and 2i + 2 for the right child. To get the parent of a node, divide the index by two and round down.

Heap Sort

- A heap is a specific kind of binary tree. First, the binary tree must be complete, such that if there is an even number of nodes, then every parent must have two children. A complete binary tree can have an odd number of nodes, if and only if the parent with one child is in the rightmost position, furthest from the root (and therefore at the largest index in an array) In a max heap, the largest value on the tree is stored in the root, and each parent node stores a value larger than its children. In a min heap, the root has the smallest value, and children have larger values. This is called the heap property, and a `heapify()` function is used to maintain this property as the heap is manipulated (by adding or removing nodes). Use a max heap (or min heap) to sort an array of values. Test your algorithm on a sorted array, a shuffled array, and a reversed array.

SECTION H - Comparing Algorithms

Which algorithm is faster? Which one should I use here? How do I decide? It is finally time to see time and space complexity in action. Hooray for efficiency.

Search Compare

- Write a program that compares searching algorithms, tracking elapsed time, and the number of checks made (where a check is used to assess if the target is found). Compare linear search to binary search. (Run binary search twice, start the time before the array is sorted, and again start the time after the sort. You can use the time module to measure time down to the nanosecond (apparently). Note each sort needs to be given the same input. If you are using a shuffled array of some length, make sure each algorithm gets the same shuffle. You can use this by using the copy module to pass a deep copy of the array to each one, passing by value, not reference. Test your algorithms on a sorted array, a shuffled array, and a reversed array. You can use global variables to track metrics.

Sort Compare

- Write a program that compares sorting algorithms, tracking elapsed time, and the number of comparisons made. (A comparison is one check for a swap, or merge) Choose three sorting algorithms to compare, including a recursive one (or otherwise O(nlog(n)) one) such as merge, quick, or heap. Note each sort needs to be given the same input. If you are using a shuffled array of some length, make sure each algorithm gets the same shuffle. You can use this by using the copy module to pass a deep copy of the array to each one, passing by value, not reference. Test your algorithms on a sorted array, a shuffled array, and a reversed array. You can use global variables to track metrics. Finally, please note that you do not have to sort a very large array. Pick large enough for significant results, but do not spend six hours running bubble sort on one million integers. (Yes, I did that for a class. Yes, it was required, and it actually took six hours each time. After the first night, I saw the crash in the morning, and realized I would need to keep an eye on things the entire evening. I added beeps and boops to the project so I could monitor it by ear while doing laundry. On the day the project was due, the other students were complaining that "bubble sort took thirty minutes to run". I said "it took six hours on my Mac", and so I won the complaint contest. I doubt the professor wanted me to spend all evening with bubble sort, and just did not imagine a modern (2020) laptop could actually take six hours to do any comparison sort.)

SECTION I - Animation and Automata

Now it is time to use two dimensional arrays and turtle graphics to put animations on a screen. (A two dimensional array is an array of smaller arrays.) We will be changing some of the settings on our turtle, so we can have more control over the screen. This can get pretty computationally intensive, so keep your computer on a flat surface, or elevated. Give the vents and fans the air they need. Make frequent backups of your work. You may need to revert to an older version of your code

Langton's Ant

- Invented by Chris Langton in 1986, this ant is a busy little bug. The ant lives of a grid of squares. Each square can be one of two colors, usually black or white. The ant can go north, south, east, or west, and change the color of each square it touches. When the ant touches a white square, it turns right, turns the square black, and continues to the next square. When the ant touches a black square, it turns left, turns the square white, and continues to the next square.
- Hints
    - You are going to use a two dimensional array for a grid. The grid is made of rows, which are made of ones and zeroes (for black and white squares)
    - The screen should only update after each step is taken, not between steps. Turn off the tracer.
    - Have the ant start in the center. Use the turtle for the ant.
    - Clear the screen every thousand steps so the ant stays fresh and happy. Otherwise, your program will slow down. The ant will remember its steps via the grid.
    - Do not use the grid render used for Game of Life. Map the grid squares directly to canvas pixels. (This ant does not care nearly as much about specific squares at particular locations. They are happy to wander about the screen.)
    - When the ant wanders too far away, reset, or terminate the program. Handle your errors, rather than crashing the program. You can completely reset the grid if you want.
    - The ant is going to wander all over the place for about ten thousand steps. It should eventually build a bridge to the edge of the screen. You will see a distinct repeating pattern characteristic of a correct Langton's Ant.

Sort Visualizer

- This is a program that animates sorting algorithms on arrays of 100 items using a bar graph animated in turtle graphics. Demonstrate any of the following sorts: Bubble, Selection, Insertion, Heap, or Quick. You can also throw in a jocular algorithm if you wish.
- Hints
    - You are going to want to turn the tracer off. Only update the screen after a comparison is made.
    - Clear the screen before each render, so the turtle stays fresh and happy (and fast).
    - Focus on implementing sorting algorithms in-place, do not use extra space.
    - The verifier function is your friend, have it check your sort when it completes. (If quick sort is having doubts, the verifier can help.)
    - Before sorting, use an in-place shuffle, that randomly swaps items 200 times (or 2n times)
    - Add a caption on your screen, describing what is happening. You can also use it to display the number of swaps made, or the number of renders made.

Conway's Game of Life

- To play Conway's Game of Life, you put cells on a grid and watch them bounce around. Each cell has eight neighbors, by which they live or die. A cell is content with two or three live neighbors. Any fewer, and it dies of isolation. Any more, and it dies of crowding. When a dead cell has three live neighbors, it comes to life! Use turtle graphics to implement this 1970 game by the late, great, John H. Conway.
- Hints
    - You are going to use a two dimensional array for a grid. The grid is made of rows, which are made of ones and zeroes (for live and dead cells)
    - To determine if a cell is alive or dead, check all eight of its neighbors: to the north, south, east, west, northeast, northwest, southeast, and southwest (or find a faster way).
    - A cell in the north-west corner is at `grid[0][0]` or at "A0". "B0" is east of "A0" and "A1" is south of "A0". "AA0" is east of "Z0". "AB0" is east of "AA0"
    - The turtle should not update constantly. Turn off the tracer.
    - The screen should only update when a generation of the game has been fully rendered, and every cell updated.
    - Clear the screen before each render so the turtle stays fresh and happy. Otherwise, your program will slow down.
    - You can assume all cells beyond the grid are dead, or you can use the modulus operator to wrap your grid around a torus (connecting the opposite edges).
    - You know your game works when you have a glider, which is a "spaceship" that travels diagonally. Your glider can have live cells at "B2", "C3", "D1", "D2", and "D3" (or anywhere else if the shape is correct).

Acknowledgements
Oh good, you made it to the end of the course. Congratulations! Did you know that the term "bug" in computing comes from an actual bug? Not really, but that does make for a good story. Legend has it that in 1947 Admiral Grace Hopper was working with the Mark II, when the computer was acting strangely. The team of engineers found an actual moth inside the computer, and taped it into the log book: "first actual case of a bug being found." I, for one, thought this story was the origin of the term for years, until the time of writing this note. Here's an article: https://daily.jstor.org/the-bug-in-the-computer-bug-story/

Of course, I have friends, family, and teachers to thank for helping me along my coding journey, and for their guidance as I put this course together over a particularly harsh winter. I started programming with the Scratch programming language at about age twelve, and transitioned to learning some python and java before completing a degree at Skidmore College. By my senior year in college I was tutoring children in computer programming. In total I have tutored in theater, algebra, reading comprehension, and writing, for just over five years. It is a joy to help students learn to achieve their goals with confidence.

Thank you for your time, and for completing this course. I sincerely hope you found it to be helpful. I named this course "Code Redstart" after the American Redstart, which is a tiny warbler that migrates through the northeastern United States. At the time of writing, I have yet to get a good picture of one. After taking over ten thousand photos of birds, I have exactly one picture of a female redstart, and it is blurry. Here is hoping for Spring of 2026.

A winner is you!